

Algoritmos y Estructura de datos III — Año 2026 – 2do Periodo

Prof. Cristian Cappelletti (TQ) / Prof. Luis Moré (TR) / Prof. Gustavo Galeano (TS)

Trabajo Práctico 1 – U1 (POO y TAD en Java)

Fecha de publicación: viernes 14/08/2026 **Fecha Entrega Límite:** miércoles 26/08/2026 23:59 hs. **Defensa oral:** jueves 27/08/2026 (Semana 4, en clase) **TP = 100 pts** (Ejercicio 1: 50p · Ejercicio 2: 25p · Defensa oral: 25p)

Cada archivo fuente deberá tener la identificación del grupo (`g_t[q,r,s]##`) y los integrantes del grupo con la indicación del número de tarea. Por cada integrante colocar: apellido, nombre, CIC y sección.

Debe incluir la Declaración de Honor con el texto mostrado en la primera clase.

⚠ **La defensa oral es individual y es requisito para todo el TP.** Cualquier integrante puede ser interrogado sobre cualquier parte del código. Un integrante que no pueda explicar el código que su grupo entregó recibe **0 (cero) en el trabajo práctico completo** — no solo en la componente de defensa —, sin afectar a los demás integrantes. Entregar código que no se puede defender equivale a no entregarlo.

Contexto y motivación

Cuando ustedes escriben en el medio de un documento, el editor tiene que insertar un carácter en una posición intermedia de una secuencia. Si esa secuencia estuviera guardada en un arreglo compacto, insertar en la posición i obligaría a correr un lugar a la derecha **todo** lo que está de i en adelante. En un archivo de un millón de caracteres, escribir una letra en el medio costaría medio millón de movimientos. Y eso por cada tecla que se aprieta.

Sin embargo, cualquier editor de texto responde instantáneamente. La estructura que lo hace posible se llama **buffer con hueco** (*gap buffer*), la usa Emacs desde 1975 y todavía hoy la usan varios editores modernos. La idea es sencilla y elegante: en vez de mantener el arreglo compacto, se deja **a propósito** un espacio vacío justo en la posición del cursor.

Escribir ahí no es insertar: es simplemente ocupar una celda que ya estaba libre. Nadie se mueve.

El precio de esa comodidad es que, cuando el usuario se va a escribir a otra parte del documento, hay que **mudar el hueco** hasta la nueva posición. Pero como una persona escribiendo se queda un buen rato en la misma zona del texto, ese costo se paga muy de vez en cuando. Esa es la intuición central del trabajo, y es la que van a tener que poder explicar en la defensa.

En este TP van a construir esa estructura y, encima de ella, el **Ctrl+Z / Ctrl+Y de un editor**: un sistema de deshacer y rehacer que revierte las operaciones de a una, carácter por carácter.

Los dos ejercicios están **encadenados**: el Ejercicio 2 opera sobre la estructura del Ejercicio 1. El Ejercicio 1 es el corazón del trabajo y concentra el puntaje; conviene terminarlo y probarlo bien antes de empezar el segundo.

Sugerencia de cómo encarar el trabajo

Una secuencia razonable para el trabajo:

Días	Objetivo
1–2	Entender el invariante. Dibujarlo en papel y trazar a mano la tabla de la sección “Ejemplo de traza” antes de escribir una línea de código.
3–4	BufferGap con insertar, borrar y get. Sin crecimiento todavía: usar un arreglo grande y fijo para no mezclar problemas.
5–6	moverCursor y el crecimiento por duplicación. Acá aparecen los bugs.
7–8	Iterable, excepciones, contador de desplazamientos y TestBufferGap. Reproducir la traza y compararla con la tabla del enunciado.
9–10	Ejercicio 2: comandos, pila propia, deshacer y rehacer. Reproducir la traza obligatoria.
11–13	

Días	Objetivo
	Tabla de desplazamientos, comentarios, README y ensayo de la defensa : háganse las preguntas entre ustedes.

Un consejo que vale puntos: si a mitad de camino no pueden explicarle a un compañero por qué el hueco está donde está, paren de programar y vuelvan al dibujo. El código que se escribe sin entender el invariante funciona a veces y no se puede defender nunca.

Ejercicio 1 (50p) — BufferGap<E>

Implemente la clase genérica BufferGap<E>, que mantiene una secuencia de elementos de tipo E en un **arreglo estático** con un hueco móvil en la posición del cursor.

El invariante de la estructura

El arreglo interno está siempre dividido en tres zonas contiguas:

Invariante del buffer con hueco

Leído en palabras:

- Las celdas desde 0 hasta inicioHueco - 1 contienen los elementos que están **antes** del cursor. Para estos, el índice lógico y el físico coinciden.
- Las celdas desde inicioHueco hasta finHueco - 1 son el **hueco**. Su contenido no significa nada: puede haber restos de elementos viejos y no importa.
- Las celdas desde finHueco hasta capacidad - 1 contienen los elementos que están **después** del cursor. Para estos, el índice lógico está corrido por el tamaño del hueco.
- El **cursor** no es un campo aparte: es inicioHueco. Si se les ocurre guardar una variable cursor además de inicioHueco, van a tener dos fuentes de verdad que se van a desincronizar.

Dos identidades que deben cumplirse **siempre**, en cualquier momento y después de cualquier operación:

tamañoHueco = finHueco - inicioHueco

size() = capacidad - tamañoHueco

Regla que define el ejercicio: ningún método puede desplazar el arreglo completo, salvo al crecer. Si su insertar recorre el arreglo, no implementaron un buffer con hueco: implementaron un arreglo común con pasos de más. El contador de desplazamientos que van a construir en la clase de prueba existe justamente para poner esto en evidencia con números.

Datos internos

Dato	Descripción
<code>private E[] datos</code>	Arreglo estático de tipo E. Inicialmente de tamaño <code>TAM_INICIAL = 16</code> .
<code>private int inicioHueco</code>	Primer índice físico del hueco. Coincide con la posición lógica del cursor.
<code>private int finHueco</code>	Primer índice físico después del hueco.
<code>private long desplazamientos</code>	Contador: cuántas veces un elemento cambió de celda física desde la creación (o desde el último reinicio del contador).

Métodos

Métodos	Funcionalidad
<code>public BufferGap()</code>	Construye un buffer vacío con capacidad <code>TAM_INICIAL</code> . El hueco ocupa todo el arreglo: <code>inicioHueco = 0</code> , <code>finHueco = capacidad</code> .
<code>public void insertar(E obj)</code>	Inserta <code>obj</code> en la posición del cursor y avanza el cursor una posición. Si el hueco se agota, la capacidad se duplica. No debe desplazar los elementos que están a la derecha del cursor.
<code>public E borrar()</code>	Elimina y retorna el elemento inmediatamente anterior al cursor (equivale a la tecla <i>backspace</i>). Lanza una excepción chequeada

Métodos	Funcionalidad
	BufferVacioException si el cursor está en la posición 0.
<code>public void moverCursor(int delta)</code>	Desplaza el cursor delta posiciones (negativo hacia la izquierda, positivo hacia la derecha), trasladando los elementos necesarios de un lado del hueco al otro. Lanza una excepción no chequeada si el cursor quedaría fuera de <code>[0, size())</code> .
<code>public int posicionCursor()</code>	Retorna la posición lógica actual del cursor.
<code>public E get(int index)</code>	Retorna el elemento en la posición lógica index mediante la fórmula de traducción sin recorrer la estructura. Lanza una excepción no chequeada si index está fuera de los límites.
<code>public E set(E obj, int index)</code>	Reemplaza el elemento en la posición lógica index y retorna el anterior. Lanza una excepción no chequeada si index está fuera de los límites.
<code>public int size()</code>	Retorna la cantidad de elementos almacenados.
<code>public int capacidad()</code>	Retorna el tamaño actual del arreglo interno.
<code>public long desplazamientos()</code>	Retorna el valor actual del contador de desplazamientos.
<code>public void reiniciarDesplazamientos()</code>	Pone el contador de desplazamientos en 0.
<code>public Iterator<E> iterator()</code>	Retorna un Iterator que recorre los elementos en orden lógico , saltando el hueco. La clase debe implementar <code>Iterable<E></code> para poder usarse con <code>for-each</code> .
<code>public String toString()</code>	Retorna el contenido en orden lógico, con el carácter `

Sobre el contador de desplazamientos: cada vez que un elemento se copia de una celda física a otra —en `moverCursor` o al crecer—, el contador se incrementa en 1. `insertar` y `borrar` bien implementados **no lo tocan**: escribir en el hueco o retroceder `inicioHueco` no mueve a nadie. Este contador es la prueba objetiva de que su estructura hace lo que promete, y la van a usar en la clase de prueba.

Cómo funciona cada operación

Insertar

Insertar en el cursor es escribir en `datos[inicioHueco]` y hacer `inicioHueco++`. Eso es todo. El hueco se achica en uno por la izquierda y ningún elemento cambia de lugar. Por eso escribir texto seguido es barato: cada tecla consume una celda del hueco, y el contador de desplazamientos no se mueve.

Mover el cursor

El hueco no se mueve solo. Para correrlo una posición hacia la izquierda hay que tomar el último elemento que está antes del hueco y llevarlo a la última celda del hueco:

Mecánica de `moverCursor`

Fíjense en lo importante: el **contenido lógico no cambió**. Sigue siendo `Ho1a`. Lo único que cambió es dónde está el hueco. Mover el cursor 5 posiciones traslada exactamente 5 elementos (el contador de desplazamientos aumenta en 5); mover el cursor de una punta a la otra del documento traslada el documento entero, y es lo más caro que se le puede pedir a la estructura. Van a poder **medirlo** con su propio contador: es un experimento de una línea.

Leer un elemento cualquiera

Acá está la parte delicada. Desde afuera, el usuario ve una secuencia de elementos consecutivos y numerados desde 0. Adentro hay un agujero en el medio. Cuando alguien pide el elemento lógico número 4, hay que averiguar en qué celda física vive:

Traducción de índice lógico a físico

si `index < inicioHueco` → `datos[index]`

si no → `datos[index + (finHueco - inicioHueco)]`

Nótese que la traducción es **una comparación y una suma**: no hay que recorrer nada, y por eso `get` no requiere iterar sobre los elementos a pesar del hueco. Esta es la fórmula

que más van a usar y la que más se equivoca. Un error acá no lanza excepción: devuelve el elemento equivocado, o basura del hueco. Por eso la clase de prueba les pide imprimir `inicioHueco` y `finHueco` en cada paso: es la única forma de ver el error temprano.

Crecer cuando el hueco se agota

Cuando `inicioHueco == finHueco` ya no queda espacio y hay que duplicar la capacidad. Copiar los elementos al arreglo nuevo es directo. Lo que tienen que decidir es **dónde queda el espacio nuevo**:

Dos opciones al duplicar la capacidad

Las dos opciones cuestan exactamente lo mismo en el momento de duplicar: hay que copiar todos los elementos igual. La diferencia aparece en la operación **siguiente**. Piénsenlo antes de programar y prepárense para justificar la elección: es una pregunta de la defensa y pesa en la rúbrica.

Recorrer con `for-each`

El iterador debe entregar los elementos en orden lógico y **saltear el hueco**. Un iterador que recorre el arreglo físico de punta a punta va a devolver los restos del hueco, que además pueden ser `null` o elementos viejos ya borrados. Recuerden que para que `for-each` funcione la clase tiene que implementar `Iterable<E>`, no solo tener un método llamado `iterator()`.

Sobre la clase `Nodo`

Este ejercicio **no** usa nodos. Todo vive en un arreglo. Si su solución tiene una lista enlazada, leyeron mal el enunciado. (En el Ejercicio 2 sí van a necesitar nodos, pero para otra cosa.)

Ejemplo de traza

Antes de leer la tabla, tres aclaraciones de notación que evitan la mayoría de las confusiones:

- En la columna **Contenido lógico**, la barra `|` marca la posición del cursor. **No es un carácter almacenado**: es solo la forma de dibujar dónde está parado el cursor (igual que hace `toString()`).
- No hay columna "Cursor" porque **el cursor es `inicioHueco`**: son siempre el mismo número. Si en su traza les dan distinto, el invariante está roto.

- finHueco solo cambia cuando el hueco se mueve (moverCursor) o cuando el arreglo crece. insertar y borrar no lo tocan.

Con TAM_INICIAL = 16, partiendo de un buffer vacío:

Fase A — escribir Hola de corrido. Cada insertar ocupa una celda del hueco y avanza inicioHueco. Nadie se desplaza; finHueco queda clavado en 16.

Operación	Contenido lógico	inicioHueco	finHueco	Qué pasó por dentro
(inicial)	`	`	0	16
insertar('H')	`H	`	1	16
insertar('o')	`Ho	`	2	16
insertar('l')	`Hol	`	3	16
insertar('a')	`Hola	`	4	16

Fase B — volver atrás y editar. Acá el hueco se muda y la fórmula de traducción empieza a trabajar.

Operación	Contenido lógico	inicioHueco	finHueco	Qué pasó por dentro
moverCursor(-2)	`Ho	la`	2	14
insertar('X')	`HoX	la`	3	14
get(4) → 'a'	`HoX	la`	3	14
borrar() → 'X'	`Ho	la`	2	14

Nota: la fila de borrar() es solo una demostración de esa operación dentro de esta traza; no es el estado que se arrastra al Ejercicio 2. El estado que continúa en el Ejercicio 2 es el de la fila anterior, HoX|1a (inmediatamente después de get(4), con inicioHueco = 3).

El estado físico del arreglo en los dos momentos clave, dibujado celda por celda:

Después de insertar('a') Después de moverCursor(-2)

índice físico: 0 1 2 3 4 15 índice físico: 0 1 2 13 14 15

contenido: H o l a [hueco] contenido: H o [hueco] l a

└ lógico 0-3 ┘

└ 0-1 ┘

└ 2-3 ┘

inicioHueco=4, finHueco=16 inicioHueco=2, finHueco=14

Los tres estados clave, vistos por dentro:

Traza del buffer

Hagan esta verificación en papel antes de programar. Reproduzcan la Fase B a mano, celda por celda, y comprueben que les da la misma tabla. Es media hora que les ahorra dos días.

Clase de prueba

Cree la clase TestBufferGap que:

1. Instancie un BufferGap<Character> y reproduzca **exactamente** la traza de las tablas anteriores, imprimiendo después de cada operación el contenido, inicioHueco, finHueco, la capacidad y el contador de desplazamientos. La salida tiene que coincidir con las tablas; si no coincide, hay un bug.
2. Inserte 100.000 caracteres aleatorios al final y verifique con el iterador (for-each) que la cantidad y el orden son correctos.
3. **Conteo de desplazamientos en el medio.** Con n caracteres ya cargados, ubique el cursor en n/2, **reinicie el contador de desplazamientos** e inserte 10.000 caracteres en esa posición. Reporte cuántos desplazamientos hizo la estructura. Repita para varios n con incrementos de 100.000:

N Desplazamientos BufferGap Desplazamientos arreglo simple

100000 #,###,### #,###,###

200000 #,###,### #,###,###

.. ..

1000000 #,###,### #,###,###

1. Implemente también una versión ingenua (insertar desplazando todo el arreglo a la derecha, contando cada celda movida) para llenar la segunda columna.

Por qué contamos desplazamientos y no milisegundos. El tiempo de reloj depende de la máquina, de la JVM y de qué más esté corriendo; la cantidad de elementos que cambian

de celda depende **solo del algoritmo**, y es reproducible: a ustedes y al docente les tiene que dar exactamente el mismo número. La tabla es la prueba objetiva de la regla que define el ejercicio: verificar que **no se desplaza el conjunto entero** al insertar.

Qué tiene que revelar la tabla. Una de las dos columnas debería quedar en 0 (o en un valor pequeño e independiente de n) y la otra debería crecer cuando crece n . Identifiquen cuál es cuál y expliquen **en un comentario dentro del código** por qué pasa eso, en términos de qué celdas toca cada versión al insertar. Pistas para autocorregirse: si la columna del BufferGap crece con n , lo más probable es que el hueco esté quedando al final al duplicar, o que moverCursor esté reconstruyendo el arreglo. Y si les da 0 también en la versión ingenua, el contador no está contando.

Nota de evaluación: el análisis asintótico formal (notación Big-O, cotas formales de complejidad) es contenido de la Unidad 2 (Semana 3) y **no constituye un requisito de entrega ni motivo de penalización** en este TP. La evaluación se centra en la **comprensión intuitiva y empírica**: explicar a partir de la tabla qué versión realiza desplazamientos y cuál opera directamente sobre el hueco sin mover elementos.

Sobre las excepciones

Deben definir al menos dos clases:

- BufferVacioException, **chequeada** (extiende Exception): la lanza borrar() cuando el cursor está en 0.
- PosicionInvalidaException, **no chequeada** (extiende RuntimeException): la lanzan get, set y moverCursor ante un índice fuera de rango.

Justifiquen en un comentario por qué cada una es de ese tipo. El criterio que se espera que discutan: una excepción chequeada representa una situación que el código que llama *puede prever y manejar razonablemente* (borrar de un buffer vacío puede pasar en un uso normal); una no chequeada representa un **error de programación** del que llama (pedir el índice 50 de un buffer de 5 elementos es un bug, no una situación a manejar). No hay una única respuesta correcta, pero sí justificaciones mal argumentadas.

Observaciones

- La solución debe alzarse en el sitio indicado en el aula virtual.
- Debe incluir los comentarios en los archivos fuentes.

- Se considerará el orden (espaciado, indentación, comentarios, etc), estilo (nombre de variables y funciones, comentario en los métodos y clases, etc) y datos (lo requerido).
- Se tendrá en cuenta el correcto uso de las estructuras de datos y de la aplicación de los conceptos de TAD.
- **NO puede usar ninguna estructura de datos del API de Java, debe implementar todas las que necesite (esto invalidará su ejercicio).** Queda prohibido ArrayList, LinkedList, Stack, ArrayDeque, Arrays.copyOf, System.arraycopy y cualquier clase de java.util.* salvo Iterator, Iterable, Random y Scanner.

Por qué esta restricción: el objetivo de la Unidad 1 es que implementen el TAD, no que lo consuman. System.arraycopy está prohibido justamente porque esconde el desplazamiento que queremos que ustedes vean y cuenten.

Rúbrica — Ejercicio 1

Rúbrica	Puntaje
Estructura BufferGap con sus campos y constructor; crecimiento por duplicación con la ubicación del hueco correctamente decidida y justificada	10
insertar y borrar operando sobre el hueco, sin desplazar el resto del arreglo	10
moverCursor con traslado correcto de elementos a través del hueco, en ambos sentidos	8
get y set con traducción correcta de índice lógico a físico (sin recorrer el arreglo)	8
Implementación de Iterable/Iterator que recorre en orden lógico omitiendo el hueco	5
Clase TestBufferGap de acuerdo a la especificación, incluidos el contador de desplazamientos, la tabla comparativa y su comentario	6

Rúbrica	Puntaje
Uso correcto de excepciones propias (una chequeada y una no chequeada, con justificación)	2
Orden, estilo, comentarios y datos	1
Total	50

Ejercicio 2 (25p) — HistorialEdicion: deshacer y rehacer

Sobre el BufferGap del Ejercicio 1, implemente el **Ctrl+Z / Ctrl+Y** de un editor: cada operación individual sobre el buffer se puede deshacer, y lo deshecho se puede rehacer. Una operación por vez, carácter por carácter.

La idea

Cuando aprietan Ctrl+Z en un editor, la última operación se revierte: si escribieron una letra, desaparece; si borraron una, reaparece. Y si aprietan Ctrl+Y (rehacer), la operación deshecha vuelve a aplicarse. Para que eso funcione, el programa necesita **recordar qué operaciones se hicieron y cómo se revierte cada una**.

Interface Comando

El mecanismo se apoya en un patrón de diseño clásico: en vez de ejecutar las operaciones directamente, se las envuelve en objetos que saben hacerse y deshacerse.

```
public interface Comando {  
  
    void ejecutar();  
  
    void deshacer();  
  
    String descripcion();  
  
}
```

Implemente los tres comandos concretos sobre un `BufferGap<Character>`:

`ComandoInsertar`, `ComandoBorrar` y `ComandoMoverCursor`.

Cada comando debe guardar el estado mínimo necesario para poder revertirse. Esto es más sutil de lo que parece y conviene pensarlo antes de programar:

- `ComandoInsertar` sabe qué carácter insertó, así que deshacerlo es borrar uno.
- `ComandoBorrar` **tiene que guardar el carácter que borró**. Si no lo guarda, no hay forma de reponerlo: el dato se perdió. Este es el error más común del ejercicio.
- `ComandoMoverCursor` tiene que poder volver el cursor a donde estaba. Decidan si guardan el desplazamiento o la posición previa, y sepan justificar por qué.

PilaES<E>

Implemente su propia pila enlazada genérica con `apilar`, `desapilar`, `tope`, `estaVacia` y `size`. La clase `Nodo` debe ser una *inner class* privada de `PilaES` — privada porque el nodo es un detalle de implementación que el usuario de la pila no tiene por qué ver.

HistorialEdicion

Dos pilas: una de **deshacer** y una de **rehacer**, ambas instancias de su `PilaES`.

Métodos	Funcionalidad
<code>public void ejecutar(Comando c)</code>	Ejecuta <code>c</code> , lo apila en la pila de deshacer y descarta la pila de rehacer .
<code>public boolean deshacer()</code>	Deshace el último comando ejecutado y lo pasa a la pila de rehacer. Retorna <code>false</code> si no hay nada que deshacer.
<code>public boolean rehacer()</code>	Vuelve a ejecutar el último comando deshecho y lo devuelve a la pila de deshacer. Retorna <code>false</code> si no hay nada que rehacer.
<code>public int sizeDeshacer()</code>	Retorna cuántos comandos hay en la pila de deshacer.
<code>public int sizeRehacer()</code>	Retorna cuántos comandos hay en la pila de rehacer.

Por qué ejecutar descarta la pila de rehacer: si el usuario deshace dos pasos y después escribe algo nuevo, el futuro que había deshecho ya no es alcanzable — se bifurcó la historia. Dejar la pila de rehacer viva permitiría “rehacer” hacia un estado que ya no existe. Este comportamiento es el de cualquier editor real: prueben en el que quieran.

Traza obligatoria a reproducir

Partiendo del buffer en el estado `HoX|1a` (traza del Ejercicio 1, fila de `get(4)`, cursor en 3 — no el resultado del `borrar()` de demostración que le sigue en esa misma tabla). Las operaciones `Insertar`, `MoverCursor` y `Borrar` de la tabla se ejecutan siempre a través de `historial.ejecutar(...)`; las columnas **desh.** y **reh.** son los tamaños de las pilas de deshacer y rehacer después de cada paso:

#	Operación	Contenido	desh.	reh.	Comentario
1	<code>Insertar('!')</code>	<code>`HoX!</code>	<code>la`</code>	1	0
2	<code>Insertar('?')</code>	<code>`HoX!?</code>	<code>la`</code>	2	0
3	<code>deshacer() → true</code>	<code>`HoX!</code>	<code>la`</code>	1	1
4	<code>deshacer() → true</code>	<code>`HoX</code>	<code>la`</code>	0	2
5	<code>rehacer() → true</code>	<code>`HoX!</code>	<code>la`</code>	1	1
6	<code>MoverCursor(-4)</code>	<code>`</code>	<code>HoX!la`</code>	2	0
7	<code>rehacer() → false</code>	<code>`</code>	<code>HoX!la`</code>	2	0
8	<code>deshacer() → true</code>	<code>`HoX!</code>	<code>la`</code>	1	1
9	<code>Borrar()</code>	<code>`HoX</code>	<code>la`</code>	2	0
10	<code>deshacer() → true</code>	<code>`HoX!</code>	<code>la`</code>	1	1
11	<code>deshacer() → true</code>	<code>`HoX</code>	<code>la`</code>	0	2
12	<code>deshacer() → false</code>	<code>`HoX</code>	<code>la`</code>	0	2

Cree la clase `TestHistorial` que reproduzca esta traza imprimiendo, en cada paso, el contenido del buffer y el tamaño de ambas pilas.

Presten atención a los pasos 7 y 10. El 7 verifica que ejecutar un comando nuevo descartó de verdad la pila de rehacer. El 10 verifica que `ComandoBorrar` guardó el carácter que

borró: si en su implementación reaparece otra cosa (o null), ahí está el error más común del ejercicio.

Observaciones

- Rigen las mismas observaciones del Ejercicio 1, incluida la prohibición de usar estructuras del API de Java.
- Las pilas de deshacer y rehacer deben ser instancias de **su** PilaES.

Rúbrica — Ejercicio 2

Rúbrica	Puntaje
Interface Comando y los tres comandos concretos, cada uno con deshacer() correcto y con el estado mínimo guardado	9
PilaES genérica propia con Nodo como <i>inner class</i> privada, usada para deshacer y rehacer	5
ejecutar, deshacer y rehacer correctos, incluida la invalidación de la pila de rehacer al ejecutar un comando nuevo	7
Clase TestHistorial que reproduce la traza obligatoria completa	2
Uso correcto de excepciones (desapilar de una pila vacía)	1
Orden, estilo, comentarios y datos	1
Total	25

Defensa oral (25p)

Se realiza en clase el día **27 de agosto**. Cada grupo dispone de **10 minutos**. La defensa es **individual dentro del grupo**: los docentes eligen a quién le preguntan cada punto.

La defensa es condición necesaria del TP completo. El puntaje de los Ejercicios 1 y 2 solo se acredita a cada integrante si ese integrante demuestra en la defensa que comprende el código entregado. Quien no pueda explicar el trabajo de su grupo recibe **0 en los 100 puntos del TP**, sin afectar a los demás integrantes.

Condiciones

- El código a defender es el entregado el 26/08. No se aceptan modificaciones posteriores.
- Se debe traer el proyecto compilando y listo para ejecutar.
- **Todos los integrantes deben estar presentes.** Un integrante ausente sin justificación recibe 0 en el TP.
- Los docentes pueden pedir ejecutar el código con entradas distintas a las del enunciado.

Rúbrica — Defensa

Rúbrica	Puntaje
Explica el invariante del hueco y la traducción de índice lógico a físico, sin leer el código	7
Justifica intuitivamente el funcionamiento de <code>insertar</code> , <code>moverCursor</code> y <code>get</code> , y explica la decisión sobre dónde queda el hueco al duplicar la capacidad	5
Traza en vivo una secuencia de operaciones propuesta por el docente sobre el pizarrón, indicando <code>inicioHueco</code> y <code>finHueco</code> en cada paso	5
Explica el diseño del deshacer/rehacer: qué guarda cada comando para poder revertirse y por qué se descarta la pila de rehacer	3
Responde a una modificación propuesta en el momento describiendo el cambio necesario	3

Rúbrica	Puntaje
Dominio individual: cada integrante responde por cualquier parte del código entregado	2
Total	25

Criterio de aplicación

Situación	Efecto
El integrante explica con fluidez y sin leer el código	Puntaje completo del ítem
Explica leyendo el código, pero con comprensión	Dos tercios del ítem
Sabe qué hace el código pero no por qué se diseñó así	Un tercio del ítem
No puede explicar código que su grupo entregó	0 en el TP completo para ese integrante
Se detecta que un integrante no participó del desarrollo	0 en el TP completo para ese integrante

Entrega

- **Formato:** un único archivo g_t[q,r,s]##-TP1.zip con todos los .java, más un README.md de máximo una página con las decisiones de diseño y la tabla de desplazamientos del Ejercicio 1.
- **Fecha límite:** miércoles 26/08/2026, 23:59 hs, hora del servidor de Educa.
- **Vía:** exclusivamente la plataforma Educa, en la actividad correspondiente del aula virtual. No se reciben trabajos por correo.
- No se reciben entregas pasada la fecha límite.